

Tutorial completo do projeto de automação

Instalação • PyCharm • Python • Playwright • Pytest • Git • Comparação de Excel e PDF

Objetivo: reunir em um único guia os passos que trabalhamos: configurar o ambiente, executar o projeto, corrigir caminhos e interpretador, comparar arquivos e enviar as alterações ao Git.

Nota: este material reúne os detalhes disponíveis da conversa. Quando uma configuração exata não foi confirmada, o tutorial orienta a verificar no seu ambiente em vez de presumir.

1. Estrutura e ambiente do projeto

O projeto utilizado é Automacao-de-Testes-com-Playwright-Python-e-Pytest, aberto no PyCharm em Windows. A pasta PageObjects contém os objetos de página e utilitários. O script de comparação trabalhado fica em Page\utilities\comparador_de_arquivos.py.

```
Automacao-de-Testes-com-Playwright-Python-e-Pytest/  
■■■ PageObjects/  
■ ■■■ Arquivos/  
■ ■ ■■■ base.xlsx  
■ ■ ■■■ atual.xlsx  
■ ■ ■■■ base.pdf  
■ ■ ■■■ atual.pdf  
■ ■■■ Page/  
■ ■■■ utilities/  
■ ■■■ comparador_de_arquivos.py  
■■■ ... arquivos do projeto
```

Os arquivos base são a referência; os arquivos atual são os que serão verificados. A pasta Arquivos já existe no projeto.

2. Python e interpretador do PyCharm

Você verificou o Python pelo terminal e a versão exibida foi Python 3.14.7. Em outro momento, o PyCharm apresentou problema com o interpretador. Para evitar instalar bibliotecas em um Python e executar com outro, confira o interpretador do projeto:

- No PyCharm, abra File > Settings (ou Ctrl+Alt+S).
- Entre em Project > Python Interpreter.
- Selecione o interpretador/ambiente virtual que pretende usar. Se necessário, adicione um ambiente virtual para o projeto.
- No terminal do PyCharm, confirme a versão com `python --version` e confira o caminho do Python com `where python`.

```
python --version  
where python
```

A versão precisa ser compatível com as bibliotecas do projeto. Se o projeto já possui ambiente virtual configurado, prefira utilizá-lo em vez de misturar instalações globais.

3. Instalar e verificar as bibliotecas

Use `python -m pip` para garantir que o pip pertence ao interpretador chamado por python. O projeto usa Playwright e Pytest; para o comparador de PDF, o script utiliza PyPDF2. O código de Excel pode depender de bibliotecas como openpyxl ou pandas conforme os imports existentes.

```
python -m pip --version  
python -m pip show pytest  
python -m pip show playwright  
python -m pip show pytest-playwright  
python -m pip show PyPDF2
```

Se alguma biblioteca exigida não estiver instalada, instale-a no ambiente ativo:

```
python -m pip install pytest
python -m pip install playwright
python -m pip install pytest-playwright
python -m pip install PyPDF2
```

Instale openpyxl ou pandas somente se o código de comparação os importar:

```
python -m pip install openpyxl pandas
```

4. Instalar o navegador do Playwright

Você executou a instalação do Chromium pelo Playwright. O comando usado foi:

```
python -m playwright install chromium
```

Esse comando baixa o navegador que o Playwright controla. Execute-o usando o mesmo interpretador/ambiente em que o pacote Playwright está instalado.

5. Executar testes com Pytest e Playwright

Foi executado um comando com --headed e browser Chromium, mas o pytest informou que coletou zero itens. Isso significa que não encontrou testes para executar; não significa que os testes passaram.

```
python -m pytest --headed --browser chromium teste.py
```

Para investigar quando aparecer “collected 0 items”:

- Confirme que está no diretório correto e que o arquivo de teste existe.
- Confira se o nome do arquivo segue o padrão test_*.py ou *_test.py.
- Confira se as funções de teste começam com test_.
- Confirme que o arquivo contém testes e não apenas funções auxiliares.
- Execute pytest com -v para ver melhor a coleta.

```
python -m pytest -v
```

Exemplo mínimo de teste Playwright, caso precise validar a instalação (crie um arquivo test_exemplo.py):

```
from playwright.sync_api import Page

def test_abre_pagina(page: Page):
    page.goto("https://example.com")
    assert page.title() == "Example Domain"
```

O exemplo depende do fixture page fornecido pelo pytest-playwright. Se o seu projeto usa outra estrutura, adapte ao padrão que já existe nele.

6. Comparação de arquivos: organização

O utilitário trabalhado fica em PageObjects\Page\utilities\comparador_de_arquivos.py. Os documentos usados nos testes ficam em PageObjects\Arquivos. A estrutura esperada para comparar os dois formatos é:

```
PageObjects/
  ■■■ Arquivos/
  ■ ■■■ base.xlsx
  ■ ■■■ atual.xlsx
  ■ ■■■ base.pdf
  ■ ■■■ atual.pdf
  ■■■ Page/
  ■■■ utilities/
  ■■■ comparador_de_arquivos.py
```

Os PDFs de exemplo criados para você possuem o mesmo texto. Eles servem para validar o cenário de igualdade.

7. Como o script encontra a pasta Arquivos

```
from pathlib import Path

PASTA_PROJETO = Path(__file__).resolve().parents[2]
PASTA_ARQUIVOS = PASTA_PROJETO / "Arquivos"
```

`Path(__file__)` representa o arquivo Python atual; `resolve()` obtém o caminho absoluto; `parents[2]` sobe duas pastas a partir do arquivo; e a segunda linha acrescenta o nome Arquivos. Confira essa lógica se mover o script para outra pasta.

8. Comparação de Excel

O bloco principal usado para comparar as planilhas é:

```
if __name__ == "__main__":
    print("Iniciando comparação dos arquivos Excel...")
    comparar_arquivos_excel(
        arquivo_base=str(PASTA_ARQUIVOS / "base.xlsx"),
        arquivo_atual=str(PASTA_ARQUIVOS / "atual.xlsx"),
    )
    print("Comparação concluída: as planilhas são iguais!")
```

Execute no terminal, estando dentro de PageObjects:

```
python .\Page\utilities\comparador_de_arquivos.py
```

Você obteve a mensagem “Comparação concluída: as planilhas são iguais!”. Isso indica que a função terminou sem levantar erro para os arquivos testados. A verificação é limitada ao que a função implementa; não presume que inclui formatação se o código não a verifica.

9. Comparação de PDF

A função `converter_e_comparar_pdf` extrai texto dos PDFs e encaminha esse conteúdo para comparação. Use este bloco principal para testar os dois documentos:

```
if __name__ == "__main__":
    print("Iniciando comparação dos PDFs...")
    converter_e_comparar_pdf(
        arquivo_base=str(PASTA_ARQUIVOS / "base.pdf"),
        arquivo_atual=str(PASTA_ARQUIVOS / "atual.pdf"),
    )
    print("Comparação concluída: os PDFs são iguais!")
```

Execute o mesmo comando no terminal. Você obteve “Comparação concluída: os PDFs são iguais!”. O comparador verifica texto extraído, não o layout visual. PDF digitalizado como imagem pode exigir OCR.

10. Executar Excel e PDF juntos

Você também executou ambos na mesma execução e recebeu mensagens de conclusão para Excel e PDF. Para isso, mantenha apenas um bloco `if __name__ == '__main__'` e coloque as duas chamadas nele:

```
if __name__ == "__main__":
    print("Iniciando comparação dos arquivos Excel...")
    comparar_arquivos_excel(
        arquivo_base=str(PASTA_ARQUIVOS / "base.xlsx"),
        arquivo_atual=str(PASTA_ARQUIVOS / "atual.xlsx"),
    )
    print("Comparação concluída: as planilhas são iguais!")

    print("Iniciando comparação dos PDFs...")
    converter_e_comparar_pdf(
        arquivo_base=str(PASTA_ARQUIVOS / "base.pdf"),
        arquivo_atual=str(PASTA_ARQUIVOS / "atual.pdf"),
    )
    print("Comparação concluída: os PDFs são iguais!")

python .\Page\utilities\comparador_de_arquivos.py
```

11. Teste negativo: comprovar que detecta diferenças

Não basta testar somente arquivos iguais. Faça também um teste com diferenças:

PDF: altere em atual.pdf uma informação como Total: 150,00 para Total: 200,00, salve e execute. O comparador deve apontar diferença ou interromper com erro, conforme a implementação.

Excel: altere uma célula correspondente em atual.xlsx, salve e execute. Confira se o comparador detecta a alteração.

Se a função continuar informando igualdade, revise quais páginas, abas, células e campos ela compara. O resultado depende das regras implementadas no script.

12. Caminhos e erros comuns

Arquivo não encontrado: confira a pasta Arquivos, o nome do documento e o diretório atual do terminal.

Se o terminal estiver dentro de PageObjects:

```
python .\Page\utilities\comparador_de_arquivos.py
```

Se estiver na raiz do repositório:

```
python .\PageObjects\Page\utilities\comparador_de_arquivos.py
```

Pytest coletou zero itens: verifique o padrão do nome do arquivo e das funções de teste, e se o arquivo contém testes executáveis.

Biblioteca ausente: instale usando `python -m pip` no ambiente correto.

PDF sem texto: documento pode ser imagem digitalizada; extração textual simples não é OCR.

13. Git: branch, commit e envio

Você criou/trabalhou na branch `comparacao-arquivos`. O Git também mostrou um alerta de “dubious ownership” ao acessar o repositório. Esse alerta é uma proteção do Git quando o dono da pasta parece diferente do usuário atual. Só marque o diretório como seguro se você confia nele e sabe que é o seu repositório.

Comandos comuns para verificar o estado:

```
git status
git branch
git remote -v
```

Para adicionar, criar commit e enviar:

```
git add .
git commit -m "Implementa comparação de arquivos Excel e PDF"
git push -u origin comparacao-arquivos
```

Você confirmou que a branch estava atualizada com `origin/comparacao-arquivos` e que a árvore de trabalho estava limpa. Para conferir no GitHub, abra o repositório, selecione a branch `comparacao-arquivos` e navegue até `PageObjects/Arquivos` e `PageObjects/Page/utilities`.

14. Checklist de conclusão

- Python e interpretador do PyCharm conferidos.
- Bibliotecas exigidas instaladas no ambiente correto.
- Chromium instalado pelo Playwright.
- Testes Pytest coletados (zero itens significa que ainda falta localizar/configurar os testes).
- Comparação Excel executada.
- Comparação PDF executada.
- Teste com arquivos diferentes realizado.
- Arquivos e script conferidos no GitHub.

Resumo do que já foi confirmado na conversa: a comparação Excel e a comparação PDF foram executadas com mensagens de igualdade; o código e os arquivos foram enviados para a branch `comparacao-arquivos`; e o Git informou árvore de trabalho limpa. O próximo passo técnico é testar um par de arquivos com diferença e, separadamente,

corrigir/confirmar a coleta dos testes Pytest.